

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ЛЬВІВСЬКА ПОЛІТЕХНІКА»

Інститут комп'ютерних технологій, автоматики та метрології

Кафедра "Електронно-обчислювальних машин"



Звіт до
лабораторної роботи №1
з дисципліни «Автоматизоване проєктування комп'ютерних систем»
з теми: «Автоматизація кросплатформенного проєкту "Hello World"
за допомогою Copilot, CMake та GitHub Actions»

Виконав:

студент групи КІ-404

Федина М. А.

Роботу прийняв:

Федак П.Р.

Львів - 2025

Мета роботи

Набути практичних навичок використання інструментів автоматизованого проектування та CI/CD (Continuous Integration/Continuous Delivery) для створення та білдуння кросплатформних програмних систем. Зокрема:

1. Використати GitHub Copilot для швидкої генерації конфігураційних файлів.
2. Створити кросплатформну систему білдуння за допомогою CMake.
3. Налаштувати автоматичне білдуння проєкту для трьох різних операційних систем (Windows, Linux, macOS) за допомогою GitHub Actions.

Лістинг згенерованого файлу CMakeLists.txt:

```
cmake_minimum_required(VERSION 3.10)
project(csad2526KI404MaksymFedyna25)
set(CMAKE_CXX_STANDARD 20)
add_executable(csad2526KI404MaksymFedyna25 main.cpp
    math_operations.h
    math_operations.cpp)
# Enable testing
include(CTest)
enable_testing()
# Add GoogleTest
include(FetchContent)
FetchContent_Declare(
    googletest
    URL https://github.com/google/googletest/archive/refs/heads/main.zip
)
# For Windows: Prevent overriding the parent project's compiler/linker settings
set(gtest_force_shared_crt ON CACHE BOOL "" FORCE)
FetchContent_MakeAvailable(googletest)
# Add test executable
add_executable(unit_tests tests/unit_tests.cpp math_operations.cpp)
target_link_libraries(unit_tests gtest_main)
include(GoogleTest)
gtest_discover_tests(unit_tests)
```

Лістинг згенерованого CI-скрипту build.sh:

```
#!/bin/bash
set -e
# Remove build directory if it exists
```

```
if [ -d "build" ]; then
  rm -rf build
fi
# Create a new build directory
mkdir build
cd build
echo "--- Running Tests ---"
# Configure the project
cmake ..
# Build the project
cmake --build .
# Run tests
ctest
echo "--- CI Script Finished Successfully ---"
```

Лістинг фінального файлу ci_build.yml:

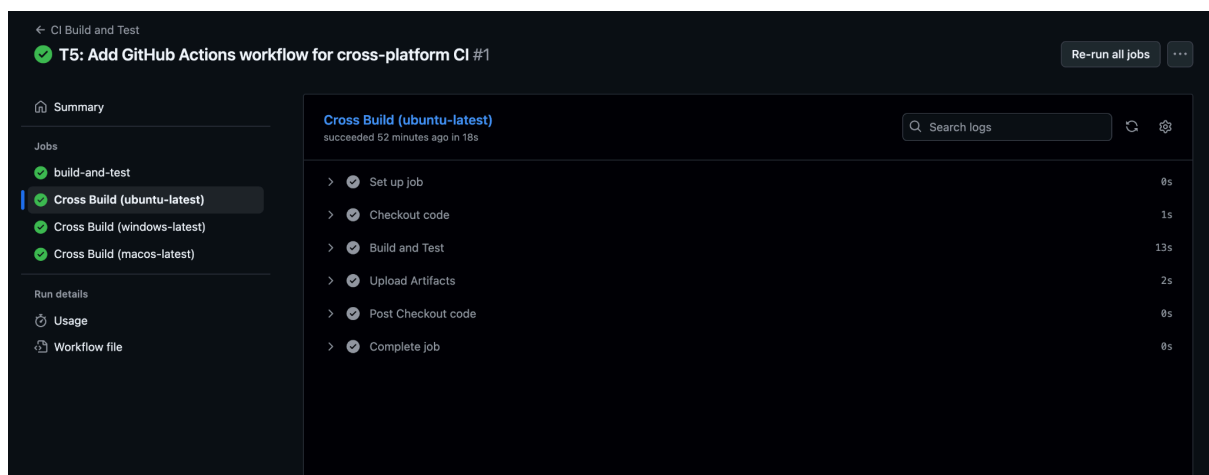
```
name: CI Build and Test
on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main, develop]
jobs:
  build-and-test:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4
      - name: Set up CMake
        uses: jwlawson/actions-setup-cmake@v2
      - name: Install dependencies
        run: sudo apt-get update && sudo apt-get install -y build-essential
      - name: Configure project
        run: cmake -S . -B build
      - name: Build project
        run: cmake --build build
      - name: Run tests
        run: ctest --test-dir build
  cross_build:
    name: Cross Build
    runs-on: ${{ matrix.os }}
    strategy:
      matrix:
```

```

os: [ubuntu-latest, windows-latest, macos-latest]
steps:
- name: Checkout code
  uses: actions/checkout@v4
- name: Build and Test
  shell: bash
  run: |
    if [[ "${runner.os}" == "Windows" ]]; then
      mkdir build
      cd build
      cmake ..
      cmake --build .
      ctest
    else
      chmod +x ./ci.sh
      ./ci.sh
    fi
- name: Upload Artifacts
  uses: actions/upload-artifact@v4
  with:
    name: build-results-${matrix.os}
    path: build/

```

Лістинг результатів виконання GitHub Actions, що підтверджують успішне білдвання на трьох ОС:



← CI Build and Test

✓ T5: Add GitHub Actions workflow for cross-platform CI #1

Re-run all jobs ⋮

Summary

Jobs

Run details

Usage

Workflow file

Cross Build (windows-latest)

succeeded 50 minutes ago in 2m 56s

Search logs 🔍 ⌛ ⚙️

> Set up job1s

> Checkout code6s

> Build and Test2m 43s

> Upload Artifacts4s

> Post Checkout code1s

> Complete job0s

← CI Build and Test

✓ T5: Add GitHub Actions workflow for cross-platform CI #1

Re-run all jobs ⋮

Summary

Jobs

Run details

Usage

Workflow file

Cross Build (macos-latest)

succeeded 52 minutes ago in 34s

Search logs 🔍 ⌛ ⚙️

> Set up job1s

> Checkout code3s

> Build and Test23s

> Upload Artifacts3s

> Post Checkout code1s

> Complete job1s

✓ T5: Add GitHub Actions workflow for cross-platform CI #1

Re-run all jobs ⋮

Summary

Jobs

Run details

Usage

Workflow file

Triggered via pull request 53 minutes ago

Status

Total duration

Artifacts

👤 maksym-fedyna-ki-404 opened #6 [feature/develop/5-add-git...](#)

Success

3m 0s

3

ci.yml

on: pull_request

✓ build-and-test1m 0s

Matrix: Cross Build

✓ 3 jobs completed

Show all jobs

Artifacts

Produced during runtime

Name	Size	Digest		
📦 build-results-macos-latest	3.46 MB	sha256:39f8588de1f6798c807798e6544636b1ce85...	📄	📥 🗑️
📦 build-results-ubuntu-latest	3.62 MB	sha256:482e44284b367035a2e80994fb1a2f9104c1...	📄	📥 🗑️
📦 build-results-windows-latest	10.0 MB	sha256:1f6c39539b5f544d907e5e9e6543b166d693...	📄	📥 🗑️

Лістинг запитів до Copilot:

Cross-Platform CI Build and Test Workflow

could you implement this task Створити вихідний файл за допомогою Copilot: У корені репозиторію створіть файл main.cpp з мінімальною програмою "Hello World": in this repo

✓ Running create_file tool

✓ Running get_errors tool

The file main.cpp has been created in the root of your repository with a minimal "Hello World" program. There are no errors in the file, so your task is complete and the program is ready to use. If you need to build or run it, let me know!

👍 💬

need to create the cmakeLists.txt and fill it with code here is the task: CMake: Метасистема білдуння, що генерує файли збірки (наприклад, makefiles, проекти Visual Studio) для різних платформ із єдиного конфігураційного файлу CMakeLists.txt. Етап 2. Генерація CMakeLists.txt

1. Створити файл: У корені репозиторію створіть файл з назвою CMakeLists.txt.
2. Використовуючи Copilot згенерувати наповнення файлу

Закomitити зміни у feature/develop/<task>, створити мердж реквест в develop, додати в ревіюера викладача.</task>

main.cpp current file CMakeLists.txt

✓ Running create_file tool

✓ Running get_errors tool

The file CMakeLists.txt has been created in the root of your repository and filled with the necessary CMake configuration to build your

Cross-Platform CI Build and Test Workflow

Create a header file for math operations. Declare a function named add that takes two integers and returns an integer.

math_operations.h current file

✓ Editing math_operations.h

The header file math_operations.h has been created with include guards and a declaration for the add function, which takes two integers and returns an integer. If you need the implementation or want to use this function in your project, let me know!

👍 💬

Implement the add function from math_operations.h

math_operations.cpp current file

✓ Editing math_operations.cpp

The add function has been implemented in math_operations.cpp. It takes two integers as arguments and returns their sum, as declared in math_operations.h. If you need to use this function in your main program or want to test it, let me know!

👍 💬

Using the Google Test framework, write unit tests for the add function from math_operations.h. Include a test case for adding two positive numbers. Include a test case for adding a positive and a negative number. Include a test case for adding two negative numbers. Include a test case for adding with zero.

unit_tests.cpp current file

Add instructions for creating a test executable file and running tests.

✓ Editing README.md

Instructions for creating a test executable and running tests have been added to your README.md. These instructions guide you through configuring the project with CMake, building the test executable, and running the tests using both ctest and by directly executing the test binary. This ensures anyone working with your project can easily build and verify the unit tests. If you need further customization or additional instructions, let me know!



Create a bash script that stops execution if any command fails. Check if a directory named 'build' exists. If it does, remove it completely. Then, create a new empty 'build' directory. Change directory into 'build', then run cmake to configure the project from the parent directory. After that, build the project using cmake. After the build is complete, run the tests using ctest.

ci.sh current file

✓ Editing ci.sh

The ci.sh script has been created. It will:

1. Stop execution if any command fails.
2. Remove the build directory if it exists.
3. Create a new, empty build directory.
4. Change into the build directory and configure the project with CMake.
5. Build the project.
6. Run all tests using ctest.

This script automates a clean build and test run for your project. If you need to make it executable, you can run:

This script automates a clean build and test run for your project. If you need to make it executable, you can run:

```
chmod +x ci.sh
```

Let me know if you need further automation or CI integration!



REM Create a windows batch script. Stop on error. REM If the build directory exists, remove it. Then create a new build directory.
REM Change into the build directory, run cmake configuration and build. REM Run tests with ctest.

ci.bat current file

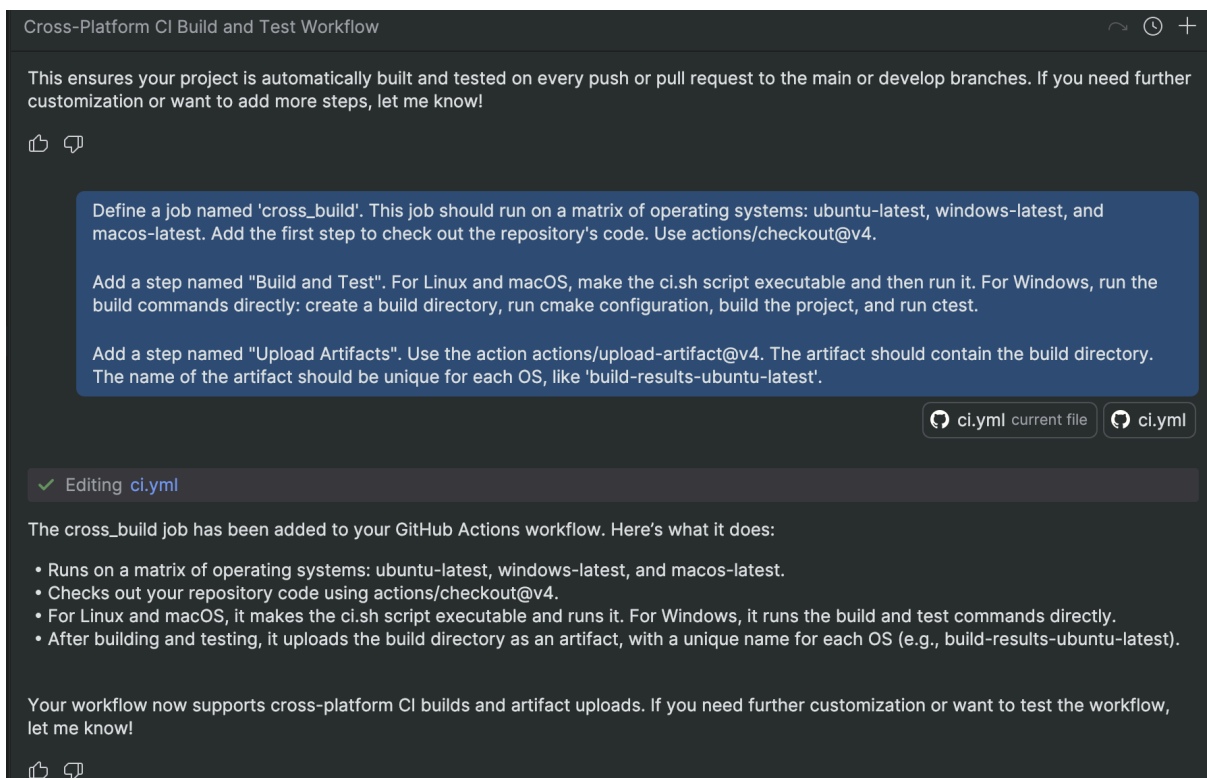
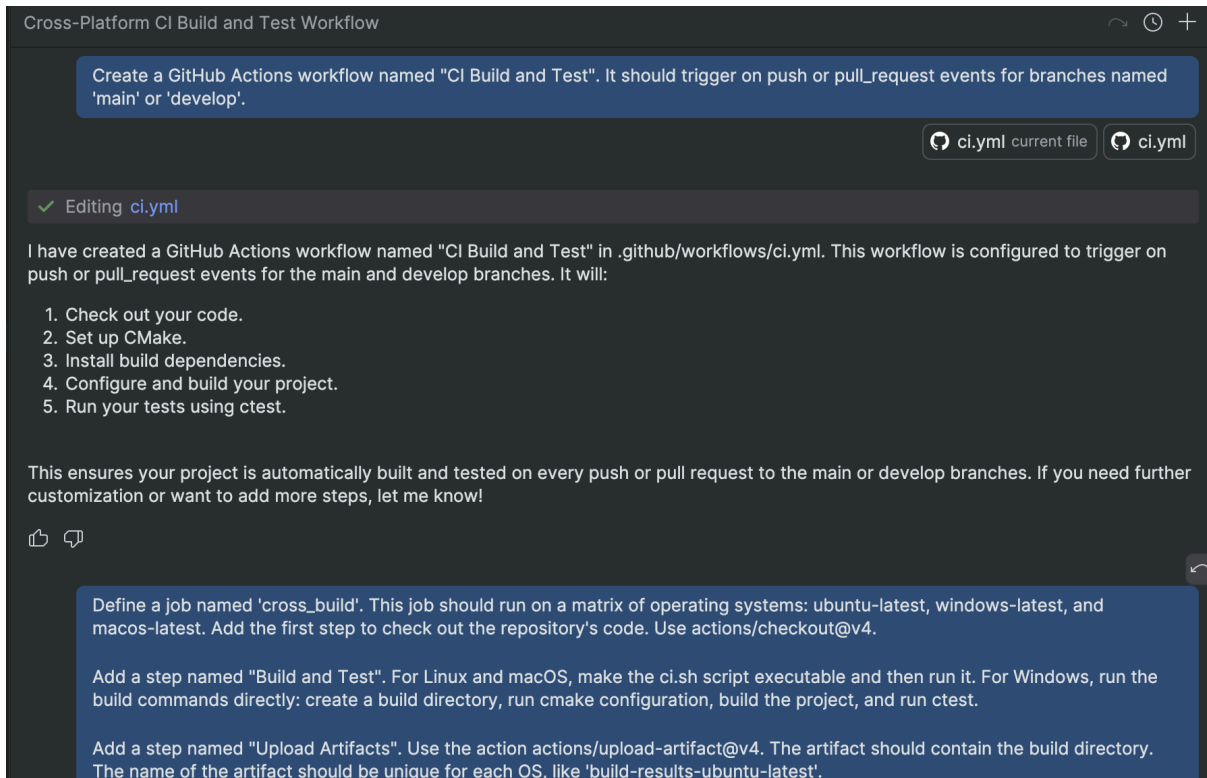
✓ Editing ci.bat

The ci.bat Windows batch script has been created. It will:

- Stop execution if any command fails (using errorlevel checks).
- Remove the build directory if it exists.
- Create a new build directory and change into it.
- Run CMake to configure and build the project.
- Run tests with ctest.

This script automates a clean build and test run for your project on Windows. If you need further adjustments or want to test the script, let me know!





Висновок: У ході виконання лабораторної роботи було створено кросплатформенний проєкт «Hello World» із використанням сучасних інструментів автоматизації — GitHub Copilot, CMake та GitHub Actions.

Використання Copilot дозволило значно скоротити час написання конфігураційних файлів і CI/CD скриптів, забезпечуючи при цьому правильність синтаксису та структури. CMake забезпечив універсальність білдування, а GitHub Actions — повну автоматизацію збірки та тестування на різних операційних системах.

Отримані навички демонструють ефективність інтеграції AI-асистентів у процес розробки програмного забезпечення, що дозволяє підвищити продуктивність, зменшити кількість помилок і полегшити реалізацію CI/CD процесів.